# Splay Trees

**A few points:**

A **Splay Tree** is a **self-adjusting Binary Search Tree (BST)**.

**Whenever a node is accessed, move that node to the root.**

This operation is called **splaying**.

There is no concept of balance factor.

**Features**:

- **No Extra Metadata:** They require **no balance accounting data** (like color bits in red-black trees or height numbers in AVL trees), saving memory.
- **Locality of Reference:** If we access a piece of data once, it moves to the top, making **subsequent lookups of the same data instantaneous**.
- **Efficiency:** While a single worst-case operation can take time proportional to the number of elements, a sequence of m operations is guaranteed to run in time proportional to m * log(n, 2).
- **Dynamic Adaptation:** The tree structure continuously morphs to fit the **actual access pattern** of the workload rather than enforcing a rigid shape.

**Uses:**

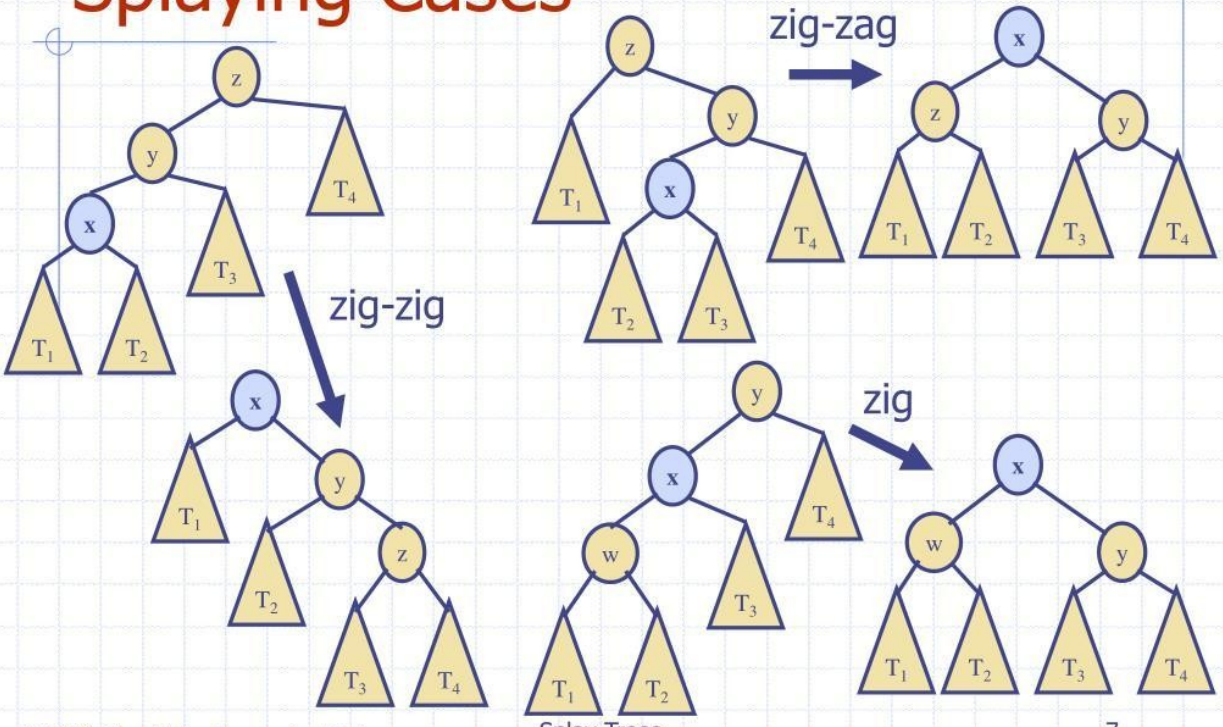
- **Caches and Memory Allocation:** Great for systems where a small subset of data is requested repeatedly.
- **Network Routers:** Used to quickly look up frequently matched IP addresses.
- **Data Compression:** Ideal for algorithms that exploit recurring patterns in data streams.

The first diagram in the next page illustrate the three basic operations : Zig-Zig, Zig-zag and Zig. We shall discuss them in detail.

The second diagram illustrates what happens when the key 2 is searched. This key will move to the root.

Frequently accessed elements move to the root. The principle of the splat tree is that the key accessed recently is likely to be accessed again in the near future.

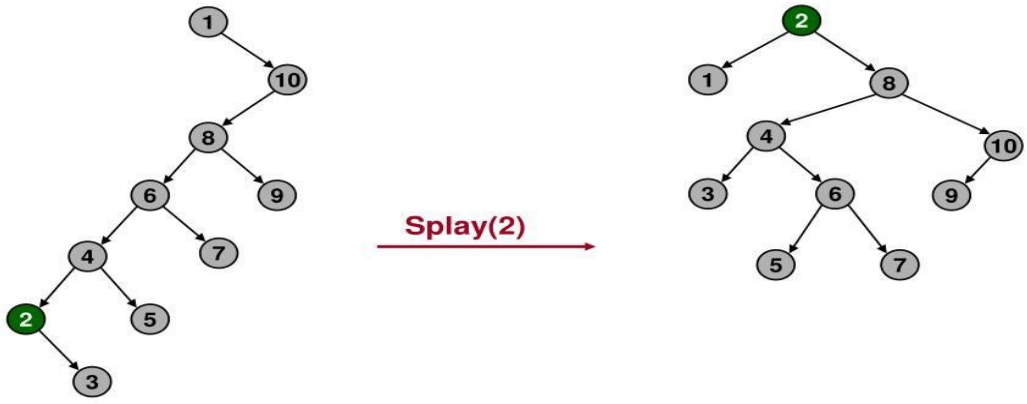
# Visualizing the Splaying Cases




© 2004 Goodrich, Tamassia, Dickerson


## Splay Example

**Apply Splay(2, S) to tree S:**



**How is a node moved to the root?**

Splaying is performed using **rotations**.

There are three basic cases:

1. **Zig**
2. **Zig-Zig**
3. **Zig-Zag**

1. Case 1 : Zig:

   This occurs when the node being accessed is a **child of the root**.

   Suppose we access 30:

```
      50
     /
    30
```

   Perform a **right rotation**:

```
     30
       \
        50
```

   So 30 becomes the root.

   Suppose we access 70

```
     50
       \
        70
```

   perform a left rotation:

```
     70
    /
```

50

**Zig rule**

Node is child of root

↓

Perform one rotation

↓

Node becomes root

---

**Case 2 — Zig-Zig**

This occurs when the node and its parent are both on the **same side**.

For example:

```
      50
     /
    30
   /
  20
```

We want to access 20.

Here:

- 20 is the **left child** of 30
- 30 is the **left child** of 50

So this is **Left-Left (LL)**.

**First rotation**

Rotate 50 right:

```
      30
     /  \
```

20 50

**Second rotation**

Rotate 30 right:

```
    20
     \
      30
       \
        50
```

So:

```
    50              20
   /                  \
  30      →           30
 /                      \
20                      50
```

This is **Zig-Zig**.

---

**5. Another Zig-Zig: Right-Right**

Consider:

```
  20
   \
    30
     \
```

50

Access 50.

Both links are to the **right**.

Therefore, it is a **Right-Right (RR)** case.

After two left rotations:

50

/

30

/

20

So:

LL → two right rotations

RR → two left rotations

---

**Case 3 — Zig-Zag**

Here the node and its parent are on **opposite sides**.

Consider:

50

/

20

\

30

We want to access 30.

Here:

- 30 is the **right child** of 20
- 20 is the **left child** of 50

Therefore:

**Left-Right (LR)** configuration.

**First rotation**

Rotate 20 left:

50

/

30

/

20

**Second rotation**

Rotate 50 right:

30

/ \

20 50

So 30 becomes the root.

---

**The four rotation patterns**

| **Configuration** | **Splay operation** |
| --- | --- |
| Node is child of root | **Zig** |

| Configuration | Splay operation |
|---|---|
| Left-Left | **Zig-Zig** |
| Right-Right | **Zig-Zig** |
| Left-Right | **Zig-Zag** |
| Right-Left | **Zig-Zag** |

A useful way to remember it:

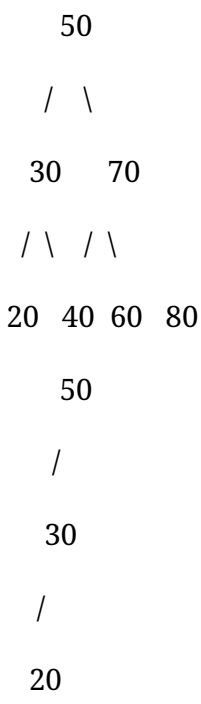
Same direction → Zig-Zig

Opposite direction → Zig-Zag

One level from root → Zig

---

Example 1: search for 20




The path is:

50 → 30 → 20

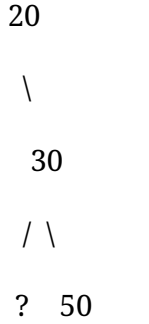
Since:

20 is left child of 30

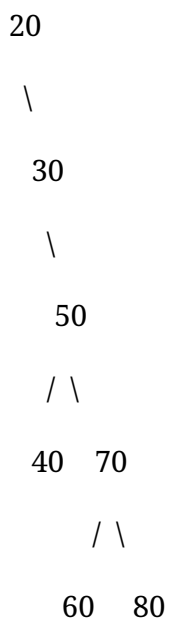
30 is left child of 50

we have **Zig-Zig**.

After splaying:



Including the remaining nodes, the resulting tree is:



The important point is not that the tree remains perfectly balanced.

The important point is:

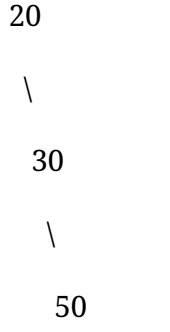
**20, which was just accessed, is now at the root.**

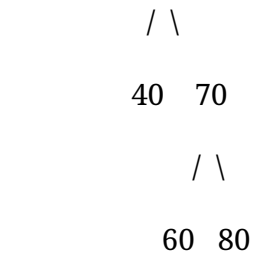
---

Example 2:

**What happens if we now access 60?**

Current tree:





The path to 60 is:

20 → 30 → 50 → 70 → 60

The node 60 is:

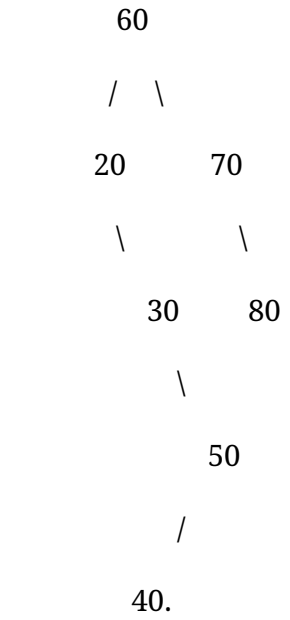
60 is left child of 70

70 is right child of 50

This is a **Zig-Zag** configuration.

After appropriate rotations, 60 moves upward.

Eventually:



and 60 becomes the root.

Thus, every access modifies the tree.

---